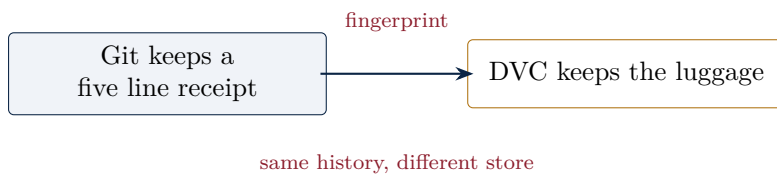


Lecture 06

The Cloakroom Receipt

Versioning data with DVC



Recap of Lecture 05 in three points. Git is the project's ledger: every change, signed and dated. Branches keep experiments safe and pull requests add review before merging. Commit what humans wrote, and ignore the data.

That last point left a hole. We told Git to ignore the `data` folder, so the dataset now has no history at all. Who versions the data?

Contents

How this handout maps to the approved session plan	4
I Why Git Alone Fails for Data	5
1 January's model cannot be rebuilt	5
2 So just commit the file to Git?	5
3 Code diffs. Data does not.	6
4 Common errors with data in Git	6
II DVC: The Receipt Stays in Git	7
5 The railway cloakroom	7
6 The three commands, and what each does	7
7 Reading the receipt	8
8 What the fingerprint does, and does not, guarantee	8
9 Common errors when handing data to DVC	9
III Switching Dataset Versions	10
10 The reviewer's question	10
11 New data, new receipt, new commit	10
12 Time travel: two commands, always together	10
13 Every commit pins an exact dataset	11
14 The lineage triple is now complete	11
15 Common errors when switching versions	12
IV The Shared Cloakroom, and Other Routes	13
16 The cache on your laptop is not a backup	13

17 DVC is not the only answer	13
V Consolidation	15
18 What our project gains today	15
19 Assignment questions, as released	15
20 Check your understanding	16
21 Vocabulary introduced today	16
VI The Week Ahead	18
22 Seven days, about seven hours	18
VII Resource Library	20
23 For this lecture specifically	20
24 Carried forward	20

List of Figures

1	The failure that data versioning exists to prevent. Note that nothing here was done carelessly: the code was committed properly, and the model is still unrecoverable. . . .	5
2	Two different materials. The conclusion is not that one is harder than the other, but that they need different stores while sharing the same history.	6
3	Where each thing lives. The receipt is five lines of text, so Git handles it happily; the file itself never enters the repository at all.	7
4	What <code>dvc add</code> actually does. Students who know only that it works tend to forget the third effect, and then wonder why the file has vanished from <code>git status</code>	8
5	The five lines, annotated. Everything the project needs to identify one exact dataset, in a file small enough for Git to carry without complaint.	8
6	The receipt difference is the data update. Not a summary of it, and not a proxy for it: those two fingerprints identify the two datasets exactly.	10
7	Why the two commands always travel together. Each moves one half of the project, and a half moved project is worse than one that never moved.	11
8	The three axes of change from the continuous delivery article, now all three under version control. When the tracking server asks which data trained this run, the receipt already knows.	11
9	The shared cloakroom. This is what the third practical adds, and it is the step that turns data versioning from a personal habit into a team capability.	13
10	What today added. Like Lecture 05, not a new box on the arc but a widening of the foundation the whole arc stands on.	15

How this handout maps to the approved session plan

Topic in this handout	Session plan location	Note
Data versioning: managing datasets and binary models beyond Git's capacity	Session 10	The core of the session
DVC	Session 10	Extended here with content addressed storage, and with what the fingerprint does and does not guarantee
The shared remote, <code>dvc push</code> and <code>dvc pull</code>	Laboratory 3	Previewed in Part IV, because the assignments stop at the local cache and students will hit sharing immediately afterwards
Schema validation and data quality	Session 13	Deliberately not here. Today versions the data; the next lecture asks whether it is any good

The taught hour, minute by minute

0 to 4	Recap, then Section 1. Ask the regulator's question and let them feel the gap
4 to 14	Sections 2 and 3, why Git is the wrong home, and why code and data are different materials
14 to 22	Sections 5 and 6, the cloakroom, and what the three commands actually do
22 to 32	Sections 7 and 8, reading the receipt and what the fingerprint means. Do not rush the fingerprint
32 to 44	Sections 10 to 12, the reviewer's question, the receipt diff, and time travel
44 to 52	Section 13, the pairing rule, and Section 14, the complete lineage triple
52 to 60	Section 16, the shared cloakroom as a preview of the laboratory, and the summary

PART I

Why Git Alone Fails for Data

1 January's model cannot be rebuilt

A food delivery company retrains monthly, and the vendor updates the orders file each time. In June a regulator asks about a prediction the model made in January.

The January *code* is in Git, one checkout away. The January *data* has been overwritten five times since.

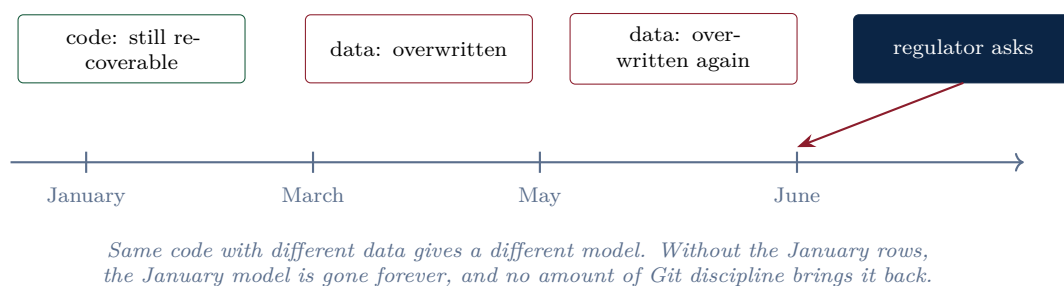


Figure 1: The failure that data versioning exists to prevent. Note that nothing here was done carelessly: the code was committed properly, and the model is still unrecoverable.

2 So just commit the file to Git?

Git keeps every version of every file forever, inside the repository. That is exactly the property you want for code and exactly the wrong one for a dataset.

What happens	Why it hurts
Ten updates of a 200 MB file	About 2 GB of history, carried by everybody
Every clone downloads all of it	A new teammate waits, and every automated job waits too
GitHub warns above 50 MiB and blocks above 100 MiB	Beyond the block your push is simply refused
Browser uploads are capped at 25 MiB	Even the manual route closes
Deleting the file today	Does not shrink the history. The blob is still in every earlier commit

GitHub's own guidance is to keep repositories ideally under one gigabyte, and strongly recommends under five. A dataset that grows monthly walks through both of those numbers without anybody noticing.

The sentence for the board

The ledger was designed for handwriting, not for luggage.

3 Code diffs. Data does not.

CODE, AS A MATERIAL

changes a few lines at a time
a difference is readable
reviewed line by line
small forever

*Text, written by a human, meant
to be read by another human.*

DATA, AS A MATERIAL

changes thousands of rows at once
a difference is meaningless noise
reviewed by checks and counts
grows every month

*Bulk, produced by a process, meant
to be summarised rather than read.*

Figure 2: Two different materials. The conclusion is not that one is harder than the other, but that they need different stores while sharing the same history.

4 Common errors with data in Git

The error	What actually happens
Committing the dataset just this once	It now lives in every clone, forever
Zipping the data into the repository	The same problem, and now unreadable as well
Emailing files to teammates	Five inboxes, five different versions each believed to be current
Cleaning the history later	Possible, but it rewrites every commit. Surgery, not housekeeping

PART II

DVC: The Receipt Stays in Git

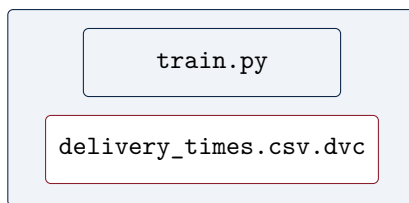
5 The railway cloakroom

At a railway cloakroom you do not carry the trunk around. You carry a paper receipt, and the trunk stays in the room.

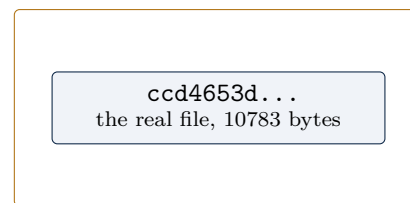
Data versioning with DVC

DVC gives Git a cloakroom. Git keeps a tiny **receipt file**; DVC stores the real data file and can always fetch back the exact version the receipt names.

GIT REPOSITORY, THE LEDGER



DVC CACHE, THE CLOAKROOM



The receipt names the data by fingerprint rather than by filename. Commit the receipt, and that commit pins that exact data, byte for byte.

Figure 3: Where each thing lives. The receipt is five lines of text, so Git handles it happily; the file itself never enters the repository at all.

6 The three commands, and what each does

```

dvc init
# Initialized DVC repository.
# You can now commit the changes to git.

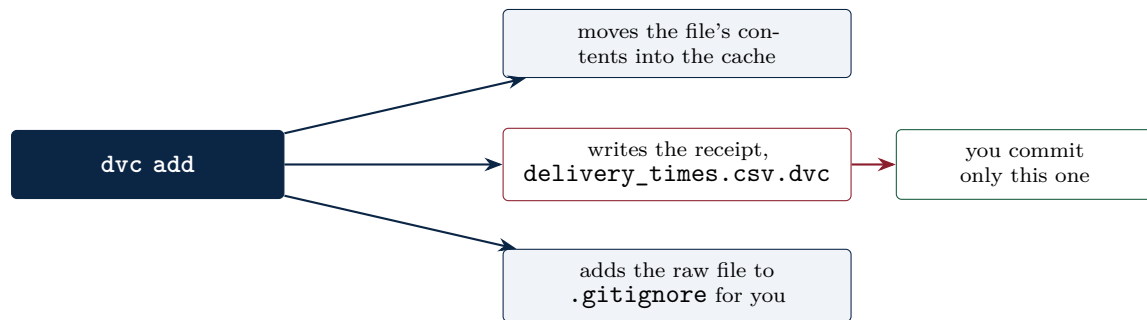
git status --short
# A  .dvc/.gitignore
# A  .dvc/config
# A  .dvcignore
  
```

One command, run inside an existing Git repository. DVC's own configuration files are small text files, so they belong in Git. Commit them and the room is open.

```

dvc add data/delivery_times.csv

# To track the changes with git, run:
#   git add 'data/delivery_times.csv.dvc' 'data/.gitignore'
  
```



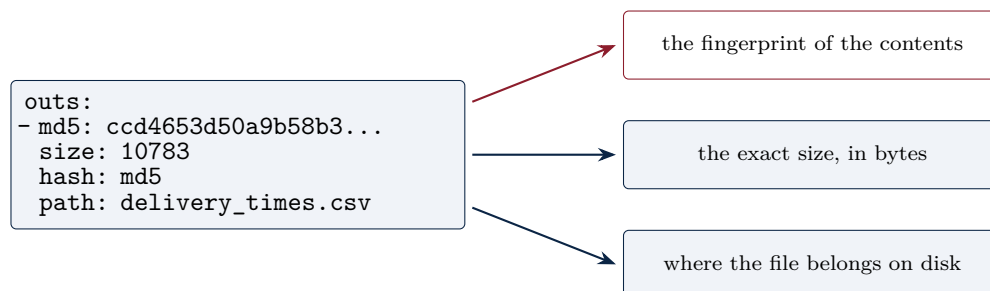
Three effects from one command, and only the middle one enters Git. The third is the quiet kindness: DVC makes it impossible to commit the raw file by accident afterwards.

Figure 4: What `dvc add` actually does. Students who know only that it works tend to forget the third effect, and then wonder why the file has vanished from `git status`.

7 Reading the receipt

```
cat data/delivery_times.csv.dvc
outs:
- md5: ccd4653d50a9b58b3d540335c0f31796
  size: 10783
  hash: md5
  path: delivery_times.csv
```

Five lines of text stand in for the whole dataset, and *this* is what you commit.



The fingerprint is what makes the receipt trustworthy. Change one byte anywhere in the file and it changes completely, so a matching fingerprint is evidence rather than a claim.

Figure 5: The five lines, annotated. Everything the project needs to identify one exact dataset, in a file small enough for Git to carry without complaint.

8 What the fingerprint does, and does not, guarantee

The fingerprint is an MD5 hash, computed from the file's contents. It changes if even one byte changes, which is exactly the property the receipt needs.

Content addressed storage, which is the idea underneath

DVC stores the file in its cache *under its own fingerprint* rather than under its name. Two consequences follow, and both are worth knowing.

First, if two versions of the dataset share unchanged content, the shared content is stored once rather than twice. Second, the same fingerprint always means the same bytes, so a receipt from a year ago still identifies exactly the file it identified then.

This is the same idea as the commit hash from Lecture 05, applied to data instead of code.

A precision worth stating to a strong class

MD5 is used here as a *content fingerprint*, not as a security measure. It is known to be breakable in the sense that an attacker can construct two different files with the same MD5, so it should never be relied on to prove that a file has not been tampered with deliberately.

For the purpose it serves here, detecting that a file has changed, it is entirely adequate, and the distinction is exactly the kind of thing an interviewer will probe.

9 Common errors when handing data to DVC

The error	What actually happens
Running <code>dvc add</code> but never committing the receipt	Git knows nothing. The version exists on your disk and nowhere in history
Forcing the raw file into Git anyway	You now pay both costs at once: a bloated repository and a receipt
Deleting the cache to save space	That is the cloakroom. Old versions die with it
Running <code>dvc init</code> outside a Git repository	DVC extends Git; it does not replace it. Initialise Git first

PART III

Switching Dataset Versions

10 The reviewer's question

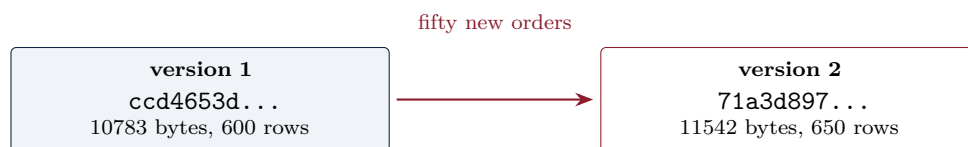
The vendor sends fifty new orders. We retrain, and the score moves. The reviewer asks the only question that matters: is the model better, or is the *data* just easier?

To answer, you must re score on the old rows. Yesterday that meant hunting for a file named something like `delivery_backup_OLD.csv`. Today it means checking out the old receipt.

11 New data, new receipt, new commit

After appending the fifty rows, run `dvc add` again and look at what Git sees:

```
git diff data/delivery_times.csv.dvc
outs:
-- md5: ccd4653d50a9b58b3d540335c0f31796
- size: 10783
+- md5: 71a3d897feb803e06cc19ad1eece1c08
+ size: 11542
```



A five line difference records a change of fifty rows. For the first time in this course, a change to the data is something a human can actually review in a pull request.

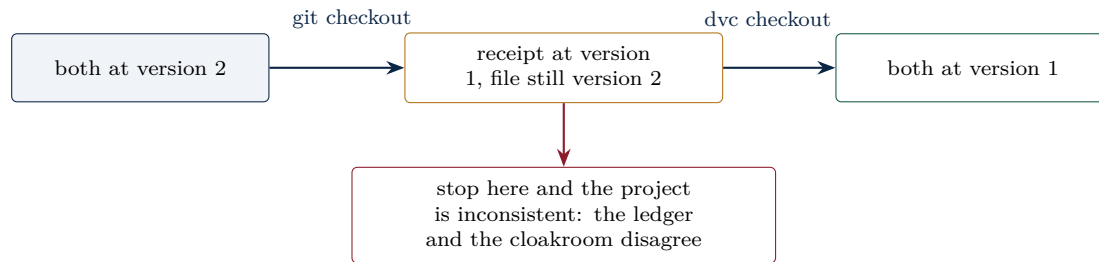
Figure 6: The receipt difference is the data update. Not a summary of it, and not a proxy for it: those two fingerprints identify the two datasets exactly.

12 Time travel: two commands, always together

```
git log --oneline
6187ffb Update dataset: 50 new orders from the vendor (650 rows)
53547ab Version the dataset with DVC (600 rows)

git checkout 53547ab    # the ledger: back to the old receipt
dvc checkout            # the cloakroom: fetch what it names
M      data/delivery_times.csv
```

The row count on disk was 650 before and is 600 after. Going forward again is the same pair in reverse.



This middle state is the single most common confusion of the practical. The code believes it is at version 1 while the file on disk is still version 2, and every score computed there is meaningless.

Figure 7: Why the two commands always travel together. Each moves one half of the project, and a half moved project is worse than one that never moved.

13 Every commit pins an exact dataset

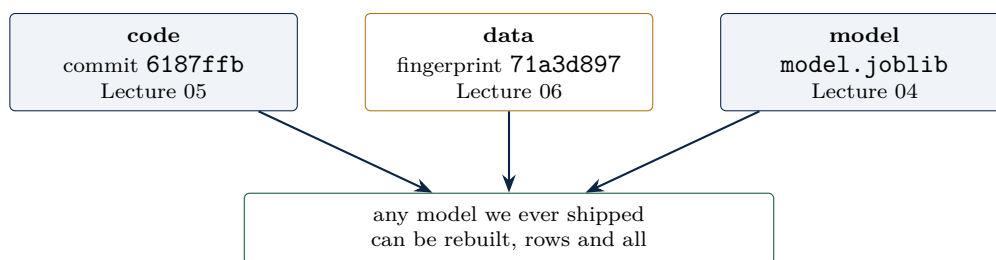
Git commit	Receipt fingerprint	Rows
53547ab	ccd4653d...	600
6187ffb	71a3d897...	650

The pairing rule

Check out any commit, run `dvc checkout`, and the code *and* its exact data are both back. January's model can be rebuilt in June.

14 The lineage triple is now complete

Three lectures have each added one piece of the same claim, and today closes it.



Remove any one of the three and the claim collapses. This is what an auditor, a regulator or an incident review is actually asking for, and it is why the next lectures can automate it.

Figure 8: The three axes of change from the continuous delivery article, now all three under version control. When the tracking server asks which data trained this run, the receipt already knows.

15 Common errors when switching versions

The error			What actually happens
git checkout checkout	without	dvc	The receipt changed and the file did not. The two halves disagree
Editing the file and forgetting add		dvc	The change exists nowhere in any history
Expecting a teammate's laptop to have your data			Sharing needs a remote cloakroom. See the next section
Comparing two models across two dataset versions			You measured the data, not the models. This is the reviewer's question, answered wrongly

PART IV

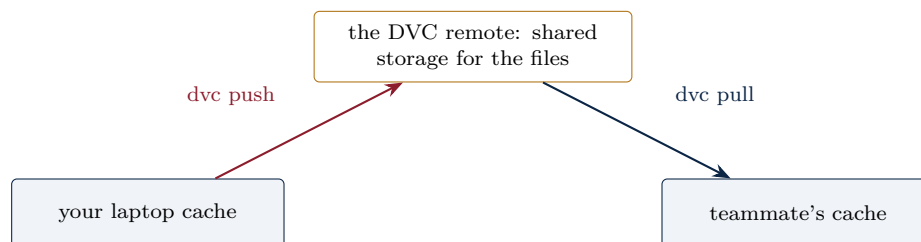
The Shared Cloakroom, and Other Routes

This part previews what the practical adds, and places DVC beside the alternative you will meet in industry.

16 The cache on your laptop is not a backup

Everything so far lives in `.dvc/cache` on one machine. Your teammate's clone has the receipt but not the file, and if your laptop dies, so does every version.

A **remote** fixes this: cloud storage, a network drive, or a shared folder, which the whole team pushes to and pulls from.



Git push and pull move the receipts; DVC push and pull move the files they name. The two pairs run side by side, and forgetting the second is why a teammate ends up with a receipt and no data.

Figure 9: The shared cloakroom. This is what the third practical adds, and it is the step that turns data versioning from a personal habit into a team capability.

```

dvc remote add -d storage <the location your instructor gives you>
git commit -m "Configure shared DVC remote"
dvc push          # send the files the receipts name
dvc pull          # fetch the files your receipts name
  
```

17 DVC is not the only answer

Git Large File Storage solves a related problem and is worth knowing, because you will meet it. GitHub's own documentation names it as the route for tracking files beyond the size limit.

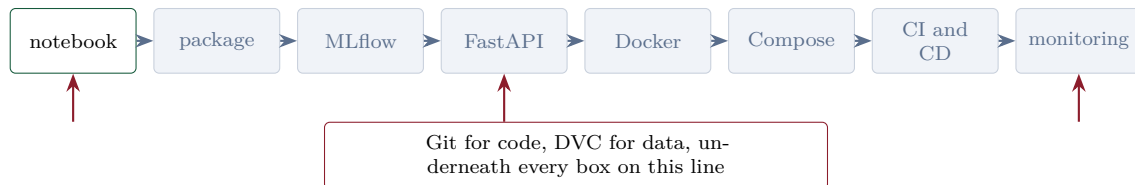
	Git LFS	DVC
What it stores elsewhere	Large files of any kind	Data and model files
Where the pointer lives	In Git, transparently	In Git, as a visible receipt file
Built for	Software repositories with big assets	Machine learning projects
Also provides	Nothing beyond storage	Pipelines, experiment tracking, data registries

The honest summary: if your problem is only that a file is too big, Git LFS is simpler. If your problem is that a model must be traceable to an exact dataset, DVC was designed for that, and this course uses it for that reason.

PART V

Consolidation

18 What our project gains today



The dataset now has versions, pinned to commits. Code and data travel together, so any model we ever ship can be rebuilt, rows and all.

Figure 10: What today added. Like Lecture 05, not a new box on the arc but a widening of the foundation the whole arc stands on.

19 Assignment questions, as released

Set 1 • Why Git alone fails for data • CO1

Q1 (Understand). Give two reasons Git alone is the wrong home for a growing dataset. Then explain why deleting a large committed file today does not make the repository small again.

Q2 (Apply). A teammate's repository takes twenty five minutes to clone: five versions of a 400 MB dataset were committed last year, then deleted. Explain in three sentences why cloning is still slow, and name the kind of tool that prevents this.

Q3 (Build). In your delivery time repository, run `git ls-files` and list what is tracked. Identify anything a new teammate could *not* regenerate from the code, write a three line note in the README saying where the data lives, and commit it with a message saying why.

Set 2 • DVC • CO1

Q1 (Understand). Using the cloakroom analogy, explain what `dvc add data.csv` puts where: what goes into the cache, what file appears next to your data, and which of the two is committed to Git.

Q2 (Apply). A receipt file contains `md5: ccd4653d...`, `size: 10783` and `path: delivery_times.csv`. Your teammate edits ten rows of the file and runs no other command. State what has and has not changed in the receipt, and why Git sees nothing to commit.

Q3 (Build). In your delivery time repository: install DVC, run `dvc init`, hand `data/delivery_times.csv` to `dvc add`, and commit the receipt. Paste the receipt's five lines and your `git log -oneline` as proof.

Set 3 • Switching dataset versions • CO1

Q1 (Understand). Explain why returning to an old dataset version needs *two* commands, `git checkout` and then `dvc checkout`, and describe what state the project is in if only the first one runs.

Q2 (Apply). A receipt difference shows the fingerprint changing from `ccd4653d...` at size 10783 to `71a3d897...` at size 11542. State what happened to the dataset, then write the exact command sequence that puts the *older* version back on disk.

Q3 (Build). In your repository from the previous assignment: append five invented orders to the file, make it version 2 with `dvc add` plus a commit, then travel back to version 1 and forward again. Paste the row counts proving both switches.

20 Check your understanding

1. In the January story, what exactly was lost, and what was still available?
2. Give three reasons a growing dataset does not belong in Git, one of which is a number.
3. Why does deleting a committed file fail to shrink the repository?
4. Name the three things `dvc add` does, and say which one you commit.
5. What does the fingerprint identify, and what would change it?
6. Explain content addressed storage in one sentence, and give one benefit.
7. Is MD5 here a security measure? Explain your answer in two sentences.
8. Your teammate edits ten rows and runs nothing else. What does `git status` show, and why?
9. Describe the state of the project after `git checkout` but before `dvc checkout`.
10. Why is a score computed in that middle state meaningless?
11. Name the three parts of the lineage triple, and the lecture that added each.
12. Your teammate clones the repository and finds a receipt but no data. What was forgotten?
13. Give one situation where Git LFS is the better choice than DVC.

21 Vocabulary introduced today

Term	Meaning
Data versioning	Recording which exact dataset a result was produced from, and being able to get it back
DVC	Data Version Control: the tool that keeps the data outside Git while the receipt stays inside it
Receipt file, <code>.dvc</code>	The small text file that names one exact dataset by fingerprint
Fingerprint	A hash computed from the file's contents, which changes if any byte changes
MD5	The hash function DVC uses for content addressing. A fingerprint, not a security measure
Content addressed storage	Storing a file under its own fingerprint rather than under its name
DVC cache	The local cloakroom, where the real files live
DVC remote	Shared storage the whole team pushes to and pulls from
<code>dvc checkout</code>	Fetches the files that the current receipts name

Term	Meaning
Pinning	A commit fixing not only the code but, through the receipt, the exact data
Lineage triple	Code commit, data fingerprint and model artifact, together
Git LFS	Git Large File Storage: an alternative for large files, without the machine learning features

PART VI

The Week Ahead

22 Seven days, about seven hours

Day 1 • The official start • 60 minutes

Read and do. Get Started with DVC, the official guide the lecture quotes. <https://dvc.org/doc/start> The documentation is also served from <https://doc.dvc.org/start>

Produce. Complete the data versioning page of the guide on a scratch repository, not yet on your project. Break it freely.

Day 2 • Your own repository • 60 minutes

Do. Assignment Set 2, Q3, on your delivery time repository: install DVC, initialise, add the dataset, commit the receipt.

Produce. The five receipt lines and your commit log. Then run `git ls-files` and confirm the raw file is no longer tracked.

Day 3 • Understand the limits you are avoiding • 30 minutes

Read. GitHub, about large files. <https://docs.github.com/en/repositories/working-with-files/managing-large-files/about-large-files-on-github>

Produce. Write down the three thresholds and the repository size recommendation, and check your own repository against them.

Day 4 • Travel in time • 60 minutes

Do. Assignment Set 3, Q3: create version 2, go back to version 1, and forward again.

Produce. The row counts at each step. Then deliberately run `git checkout` *without* `dvc checkout` and record what the row count says. Seeing the inconsistent state once is worth more than reading about it.

Day 5 • Share it • 45 minutes

Read. The DVC guide on remote storage, then configure one, even a local folder on your own machine.

Produce. A successful `dvc push`, then delete your cache and recover the file with `dvc pull`. That recovery is the whole argument for a remote.

Day 6 • The alternative • 30 minutes

Read. GitHub's page on Git Large File Storage, linked from the large files page above.

Produce. Three sentences on when you would choose it over DVC, and when you would not.

Day 7 • Close the triple • 45 minutes

Do. Train your model on the current dataset, and record three things beside the artifact: the commit hash, the receipt fingerprint, and the score.

Why this matters. You have now built the complete lineage record by hand. From Lecture 11 the tracking server will capture all three automatically, and you will understand exactly what it is capturing and why.

If you only have one hour all week

Day 2, then Day 4. Getting the dataset under version control, and watching the row count change when you travel between versions. Everything else follows from having seen that work once.

PART VII

Resource Library

All links verified on 9 September 2026.

23 For this lecture specifically

Resource	Where, and why
Get Started with DVC	https://dvc.org/doc/start The official guide, which calls DVC your Git for data. Also served from https://doc.dvc.org/start
DVC documentation home	https://dvc.org/doc Commands and user guide, including remotes and the cache
GitHub, about large files	https://docs.github.com/en/repositories/working-with-files/managing-large-files/about-large-files-on-github The 25, 50 and 100 MiB thresholds, and the repository size recommendation
Sato, Wider and Windheuser, Continuous Delivery for Machine Learning	https://martinfowler.com/articles/cd4ml.html The three axes of change. Today put the second one under version control
Chacon and Straub, <i>Pro Git</i>	https://git-scm.com/book/en/v2 For why a deleted file stays in the history, read the chapter on Git internals

24 Carried forward

Resource	Where
Sculley et al., Hidden Technical Debt in Machine Learning Systems	https://proceedings.neurips.cc/paper_files/paper/2015/file/86df7dcfd896fcdf2674f757a2463eba-Paper.pdf Unstable data dependencies are exactly today's problem
Kapoor and Narayanan, leakage and the reproducibility crisis	https://doi.org/10.1016/j.patter.2023.100804

Resource	Where
Amershi et al., Software Engineering for Machine Learning, ICSE 2019	https://www.microsoft.com/en-us/research/wp-content/uploads/2019/03/amershi-icse-2019_Software_Engineering_for_Machine_Learning.pdf On why managing and versioning data is more complex than in other software engineering
nbstripout	https://github.com/kynan/nbstripout
Machine Learning in Production, DeepLearning.AI, free to audit	https://www.coursera.org/learn/introduction-to-machine-learning-in-production
MLOps Zoomcamp, DataTalks.Club	https://github.com/DataTalksClub/mlops-zoomcamp
Made With ML	https://madewithml.com
Stanford CS 329S	https://stanford-cs329s.github.io

Git keeps the receipt. DVC keeps the luggage.

Next lecture. Lecture 07, when bad data compounds. We can now version data; the next question is whether it is any good. Data cascades and how small data problems become large failures downstream, checking a file on arrival for columns, types and ranges, and goodness of fit against goodness of data. The third practical comes first: DVC on your own repository, plus the shared remote.